DreamCatcher Study Guide

Oct 11, 2026 · @Group

1. The big picture

DreamCatcher answers one question for a university student: **based on my answers about sleep, lifestyle and health, which sleep pattern am I most like, and should I see a professional?** It is a screening aid, not a diagnosis.

- **Input:** 10 answers (gender, age, sleep duration, sleep quality, stress, physical activity, BMI category, blood pressure, heart rate, daily steps).
- **Output:** the most likely pattern (Insomnia, Sleep Apnea or No Sleep Disorder), how confident the model is, and screening advice (Higher likelihood, Possible, Lower likelihood).
- **Best model:** KNN (K-Nearest Neighbors), chosen by cross-validation.
- **Honest headline:** the model is better than guessing, but only modestly (test macro-F1 about 0.47 vs 0.20 for a no-skill baseline). That is normal for self-reported survey data, and the documentation says so openly.

If the panel remembers one sentence, make it this: *we built a careful, leakage-safe pipeline, compared three models fairly, picked KNN, and deployed it as a web app that explains its own uncertainty.*

2. The data and how it was prepared

The BD-Uni Sleep Disorder Dataset has **619 students** from five Bangladeshi universities, **12 predictors** and one target with three classes: Insomnia 265 (43%), Sleep Apnea 241 (39%), No Sleep Disorder 113 (18%).

Step	What we did	Why, in one line
Fix messy text	Trimmed spaces, unified dashes and capital letters; 38 department spellings became 5 groups	Otherwise "CSE" and "CSE " count as two departments

Step	What we did	Why, in one line
Impossible values	3 blood-pressure readings (118/26, 80/90, 80/120) set to missing, then filled with the median	A wrong reading cannot be repaired by guessing
Outliers	Age capped at 30 (2 students); heart rate and activity limited to plausible ranges	Keeps every student but stops extreme values pulling KNN and SVM
New features	Systolic, diastolic, mean arterial pressure, pulse pressure from the "120/80" text	A model cannot read text like "120/80"
Encoding	Ordered answers became 0, 1, 2…; gender one-hot; target via LabelEncoder	Models need numbers; order (Poor < Good) is kept
Scaling	StandardScaler (mean 0, spread 1)	So big numbers (minutes) do not dominate distance-based models
Split	80% training (495) / 20% test (124), stratified, random_state = 42	The test set stays untouched until the end

Leakage, in one sentence: anything that learns from the data (imputing, scaling, SMOTE, feature selection) sits inside the pipeline, so it is fitted on training data only and never peeks at the test set.

Key EDA finding to say out loud: the features are only weakly related to the target. For example, students who sleep 8+ hours are still 40% Insomnia and 40% Sleep Apnea in this survey. That is why no model can score very high.

3. The three models and how they were trained

Model	How it decides, in plain words	Best settings found
Random Forest	300 decision trees each vote; the majority wins	depth 5, at least 5 students per leaf, top 12 features, SMOTE
KNN	Finds the 25 most similar students and takes the most common class among them	k = 25, Manhattan distance, equal votes, all features, SMOTE

Model	How it decides, in plain words	Best settings found
SVM	Draws the best curved boundary between the classes	C = 1, gamma = scale, top 12 features, SMOTE

How training worked:

1. One **pipeline** per model: fill missing values, scale, encode, optional SMOTE, keep the best features, then the classifier. Same preprocessing for all three, so the comparison is fair.
2. **SMOTE** creates synthetic examples of the smaller classes (No Sleep Disorder grew from 90 to 212 in training) so the model does not ignore them.
3. **GridSearchCV** tried every combination of settings (72 for RF, 96 for KNN, 108 for SVM).
4. Each combination was scored with **5-fold cross-validation repeated twice**: training data split into 5 parts, train on 4, test on 1, rotate, do it all twice. That gives 10 scores per setting, so one lucky split cannot fool us.
5. The score used to choose was **macro-F1** (explained in section 4), because it treats the small "No Sleep Disorder" class as equally important.

Why KNN won: best cross-validated macro-F1 (0.399 vs 0.389 SVM and 0.384 RF). With weak, noisy survey answers, averaging over 25 neighbours smooths out the noise, while trees tend to memorise noise. The models are close, so say "KNN is slightly better", not "much better".

Feature set: the deployed model drops academic level and department (administrative, not medical). Performance did not get worse (test macro-F1 0.454 → 0.466), and the user answers fewer questions.

4. Reading the evaluation metrics

Every metric answers a different question. Numbers below are KNN on the 124 test students (all features), the documentation's main results table.

Metric	The question it answers	KNN	Baseline
Accuracy	Out of all students, how many did it label correctly?	0.452	0.427
Precision	When it says "Insomnia", how often is that right?	0.450	0.142

Metric	The question it answers	KNN	Baseline
Recall	Of the real Insomnia students, how many did it find?	0.479	0.333
F1-score	One number balancing precision and recall	0.454	0.200
Balanced accuracy	Average recall over the three classes	0.479	0.333
MCC	Overall agreement after removing luck (0 = chance, 1 = perfect)	0.165	0.000
ROC-AUC	Pick one student from a class and one from another: how often does it rank them correctly?	0.66	0.50

"Macro" means the score is calculated for each class separately and then averaged, so the small No Sleep Disorder class counts as much as the big Insomnia class.

The baseline always answers "Insomnia" (the biggest class). It already gets 42.7% accuracy without learning anything. That is why accuracy alone is misleading here, and why we compare macro-F1 (0.454 vs 0.200).

Reading the confusion matrix (KNN): rows are the true class, columns what the model said. KNN found 23 of 53 Insomnia students, 19 of 48 Sleep Apnea students and 14 of 23 No Sleep Disorder students. Most mistakes are Insomnia confused with Sleep Apnea, because the survey lacks the questions that separate them (snoring, observed breathing pauses).

Is the signal real? The permutation test shuffled the labels 100 times; the real score beat almost all shuffled runs ($p = 0.010$). So the model learned something real, but not much.

5. Sensitivity, specificity and the screening thresholds

Why it exists: the web app's advice (Higher likelihood / Possible / Lower likelihood) answers a yes-or-no question: *might this student have any sleep disorder?* Sensitivity and specificity measure how good that yes-or-no answer is. The requirement sheet does not ask for them, but the app's advice is built on them, so the panel may ask.

- **Sensitivity** = of the students who really have a disorder, how many did we flag? (catching real cases)

- **Specificity** = of the students without a disorder, how many did we correctly clear? (avoiding false alarms)

A memory trick: **Sen**sitivity looks at the **sick**, **sp**ecificity looks at the **healthy (no disorder)**.

The threshold is a cut-off. The model gives each student P(any disorder) = 1 − P(No Sleep Disorder). If that number is at or above the cut-off, the student is flagged. Lower the cut-off and you flag more people: you miss fewer real cases (sensitivity up) but raise more false alarms (specificity down). You cannot have both.

Worked example (test set: 101 students with a disorder, 23 without):

Cut-off	Real cases flagged	Sensitivity	Healthy students cleared	Specificity	Shown in the app as
0.69 (balanced)	52 of 101	51%	19 of 23	83%	P(any) ⩾ 0.69 → Higher likelihood
0.44 (high sensitivity)	87 of 101	86%	5 of 23	22%	0.44 ⩽ P(any) < 0.69 → Possible

Below 0.44 the app says **Lower likelihood**.

How the cut-offs were chosen: on the training data only. 0.69 gives the best balance of the two measures; 0.44 is the highest cut-off that still catches at least 90% of real cases in the training data (on the new test students it caught 86%, normal variation for 101 students).

The one-line answer if asked "why not just use accuracy?" In screening, missing a real case is worse than a false alarm, so we report how many real cases we catch (sensitivity) and what it costs in false alarms (specificity), like the STOP-Bang sleep-apnea questionnaire does.

6. The two decision rules

The model gives three probabilities; two rules decide which single answer the user sees.

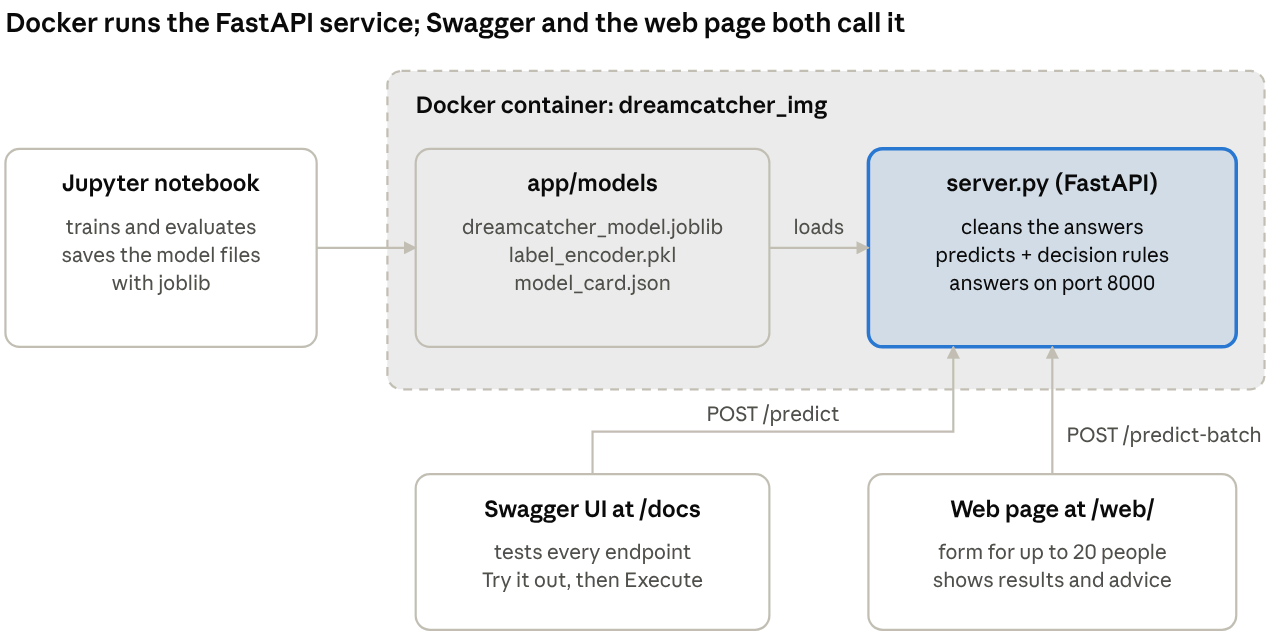
Rule 1: ties go to a disorder. KNN counts the classes of the 25 nearest students, so every probability is a multiple of 4% (1 of 25). Two classes can tie, for example Sleep Apnea 36% and No Sleep Disorder 36%. Without a rule the computer picked whichever came first

alphabetically, so it said "No Sleep Disorder". Now a tie is reported as the disorder (missing a real case is worse) and the page says it was a tie. 5 of the 124 test students had a tie.

Rule 2: no Insomnia for well-rested, satisfied sleepers. Insomnia by definition means being dissatisfied with how much or how well you sleep. A student who sleeps 8+ hours and rates quality Good or Excellent reports neither, so Insomnia is set to 0% and the other two are rescaled to add up to 100%. Example: the model said Insomnia 44%, No Sleep Disorder 44%, Sleep Apnea 12%; after the rule it shows No Sleep Disorder 79%, Sleep Apnea 21%.

Why did the test score go down with Rule 2? (macro-F1 0.474 → 0.438). Because the survey itself labels 41% of these good sleepers as Insomnia. The rule disagrees with those labels exactly where they contradict the medical definition. We chose a result that makes clinical sense over a slightly higher score, and we report the cost openly. That is a strong answer if the panel asks.

7. How deployment works



DreamCatcher deployment · notebook, container, two ways in

The notebook saves three files into `app/models`. Docker packages the `app` folder with Python and the pinned packages from `requirements.txt`; inside the container, `server.py` loads the model once and waits on port 8000. Swagger (`/docs`) is for testing

the API directly; the web page (`/web/`) is what a student uses.

What happens on one request: the answers are checked (Pydantic rejects invalid values with error 422), cleaned with the same rules as the notebook (`dreamcatcher_utils.py`), passed to the KNN pipeline, then the two decision rules pick the answer, and the result goes back as JSON.

Folder structure (the professor's standard plus three DreamCatcher items): `app/dataset` , `app/models` , `app/server.py` , `Dockerfile` and `requirements.txt` are the standard. The extras are `app/static/index.html` (the web page), `app/dreamcatcher_utils.py` (same cleaning and rules as the notebook) and `app/__init__.py` (an empty file that lets `server.py` import the utils).

Running it: in a terminal inside `dreamcatcher_app` , `docker build -t dreamcatcher_img .` then `docker run --name dreamcatcher -p 8000:8000 dreamcatcher_img` . If Docker says the name is in use, run `docker rm -f dreamcatcher` first. After changing code or retraining, build again.

8. Likely panel questions

1. **Why is your accuracy only about 45%?** The labels are self-reported symptoms, not confirmed diagnoses, and the survey lacks key clues (snoring, breathing pauses). The features are only weakly related to the target, so no model can do much better. We still beat the baseline clearly in macro-F1 (0.454 vs 0.200), and the permutation test shows the signal is real (p = 0.010).
2. **Why macro-F1 and not accuracy?** A model that always says "Insomnia" gets 42.7% accuracy while learning nothing. Macro-F1 gives each class equal weight, so the model cannot score well by ignoring the small No Sleep Disorder class.
3. **Why did KNN win?** Highest cross-validated macro-F1 (0.399). Averaging over 25 neighbours smooths noisy survey answers. The three models are close, so we call KNN slightly better, not clearly better.
4. **What is SMOTE and is it cheating?** It creates synthetic examples of smaller classes. It is applied only to training folds inside the pipeline, never to the test set, so the reported scores are on real students only.
5. **How did you avoid data leakage?** The test set was split off first and touched once. All steps that learn from data sit inside the pipeline and are fitted on training folds only.

6. **Why drop academic level and department?** They are administrative, not medical. Removing them did not hurt performance (test macro-F1 0.454 to 0.466) and the user answers fewer questions.
7. **Why does the app say "No Sleep Disorder" but advice "Possible"?** The pattern is the single most likely class; the advice uses the probability of any disorder (Insomnia + Sleep Apnea). Illustration: No Sleep Disorder 40%, Insomnia 32%, Sleep Apnea 28%. No Sleep Disorder is the top class, but any disorder is 32% + 28% = 60%, above the 0.44 cut-off, so the advice is "Possible".
8. **Why rule out Insomnia for good sleepers if it lowers the score?** See section 6: the labels contradict the definition of insomnia there; we chose clinical sense and reported the cost.
9. **How is the model deployed?** Saved with joblib, served by FastAPI, packaged with Docker, tested in Swagger at /docs, used through the web page at /web/ (section 7).
10. **What would you improve?** Validated questionnaire items (Insomnia Severity Index, STOP-Bang), clinically confirmed labels, a two-stage model (disorder vs none, then which disorder) and testing at another university.

9. Key numbers cheat sheet

What	Number
Students / predictors / classes	619 / 12 / 3
Class shares	Insomnia 43%, Sleep Apnea 39%, No Sleep Disorder 18%
Train / test split	495 / 124 (80/20, stratified, random_state 42)
Best CV macro-F1	KNN 0.399, SVM 0.389, Random Forest 0.384, baseline 0.200
Test macro-F1 (all features)	KNN 0.454, SVM 0.416, Random Forest 0.362
Test macro-F1, deployed KNN	0.466 model alone; 0.474 with tie rule; 0.438 with both rules
KNN settings	k = 25, Manhattan distance (p = 1), uniform weights, SMOTE, all features
Permutation test	p = 0.010

What	Number
Screening cut-offs	0.69 balanced, 0.44 high sensitivity
High-sensitivity result on test	catches 86% of disorder cases, clears 22% of healthy students
Deployed features	13 (academic level and department dropped)